Binary search trees solutions

12-E01 Inorder and BST validation

Derivation

Left-subtree inorder is 3,6; the root is 5; right-subtree inorder is 7,8. Complete inorder is 3,6,5,7,8.

Result and check

The sequence is not nondecreasing, and 6 lies in root5's left subtree while exceeding5, so this is not a BST. Checking only parent-child comparisons 3<5 and 6>3 misses the cross-level violation.

12-E02 Successor path

Derivation

Inorder order follows the structure. With no right child, ascend until first entering an ancestor from its left subtree. From4 to2, the current node is a right child, so continue. At6, the previous node2 is its left child, so stop.

Result and check

The successor is6. The stopping test is child==parent.left, not a numerical key comparison. If no left ascent occurs before passing the root, no successor exists.

12-E03 Two-child deletion

Derivation

If12 is not10's direct right child, replace12 at its old position with13 and set13.parent to12's old parent. Attach the original10.right as12.right and update its parent. Then replace10 with12, set12.left=10.left, and update that child's parent. If12 already equals10.right, preserve its existing right child13 and skip the earlier transplant; do not attach13 twice.

Result and check

Update the root and all relevant parent fields consistently. Inorder remains sorted,10 is removed, and13 is retained.

Binary search trees solutions (continued)

12-E04 Transplanting the successor's right child

Derivation

The successor is y25, with old parent30 and right child27. Set30.left=27 and27.parent=30; then25.right=40 and40.parent=25; root=25 and25.parent=NIL; finally25.left=10 and10.parent=25. Other edges remain unchanged. Inorder becomes5,10,15,25,27,30,35,40,50.

Result and check

The original node20 handle becomes invalid. The node25 handle still denotes the same object, now relocated. Copying25's key/data into node20 and deleting node25 invalidates a different identity and is not the same API behavior.

12-E05 Uniform permutations versus uniform shapes

Derivation

213 and231 both yield root2 with children1,3 and height1. Each of123,132,312,321 produces a different three-node chain of height2. Uniform permutations give (2×1+4×2)/6=5/3. Uniform shapes give (1+4×2)/5=9/5.

Result and check

Different models give different denominators and probabilities. Equal shape probabilities cannot be substituted into the random-insertion recurrence. In the correct model, root rank is uniform and, conditional on it, both relative subtree insertion orders are uniform.

12-E06 Average depth does not control maximum depth

Derivation

The original N nodes have total depth Θ(N log N). The added chain has depths h,h+1,…,h+L−1 and total Lh+L(L−1)/2=O(sqrt(N)log N+N). Total depth remains Θ(N log N). Since n=Θ(N), average depth is Θ(log n).

Result and check

Height is h+L−1=Θ(sqrt(n)), far larger than log n. Small average depth permits a few very deep nodes. The lecture instead controls the expectation of 2^{height} directly and then uses Jensen and Markov for the maximum.

Binary search trees solutions (continued)

Verification code appendix: bst_checks.py

```
"""Validate real pointer operations, all small insertion orders, and exact height moments."""
from itertools import permutations
from fractions import Fraction
from math import comb
import random


class Node:
    def __init__(self,key):
        self.key=key
        self.left=self.right=self.parent=None


class Tree:
    def __init__(self,keys=()):
        self.root=None
        self.events=set()
        for key in keys:
            self.insert(key)
    def search(self,key):
        node=self.root
        while node is not None and key!=node.key:
            node=node.left if key<node.key else node.right
        return node
    def insert(self,key):
        parent=None
        node=self.root
        while node is not None:
            if key==node.key:
                return node
            parent=node
            node=node.left if key<node.key else node.right
        new=Node(key)
        new.parent=parent
        if parent is None:
            self.root=new
        elif key<parent.key:
            parent.left=new
        else:
            parent.right=new
        return new
    def transplant(self,u,v):
        parent=u.parent
        if parent is None:
            self.root=v
        elif u is parent.left:
            parent.left=v
        else:
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Binary search trees solutions (continued)

```
            parent.right=v
        if v is not None:
            v.parent=parent
    def delete(self,z):
        if z.left is None:
            self.events.add('leaf' if z.right is None else 'only-right')
            self.transplant(z,z.right)
        elif z.right is None:
            self.events.add('only-left')
            self.transplant(z,z.left)
        else:
            y=z.right
            while y.left is not None:
                y=y.left
            if y.parent is not z:
                self.events.add('two-deep')
                if y.right is not None:
                    self.events.add('deep-successor-has-right')
                self.transplant(y,y.right)
                y.right=z.right
                y.right.parent=y
            else:
                self.events.add('two-direct')
            self.transplant(z,y)
            y.left=z.left
            y.left.parent=y


def neighbor(node,forward=True):
    child='right' if forward else 'left'
    other='left' if forward else 'right'
    steps=0
    if getattr(node,child) is not None:
        node=getattr(node,child)
        steps+=1
        while getattr(node,other) is not None:
            node=getattr(node,other)
            steps+=1
        return node,steps
    parent=node.parent
    steps+=1
    while parent is not None and node is getattr(parent,child):
        node,parent=parent,parent.parent
        steps+=1
    return parent,steps


def validate(tree,expected):
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binary search trees solutions (continued)

```
    seen={}
    def walk(node,parent,lo,hi):
        if node is None:
            return [],-1
        assert node not in seen.values()
        assert node.parent is parent and lo<node.key<hi
        seen[node.key]=node
        left,lh=walk(node.left,node,lo,node.key)
        right,rh=walk(node.right,node,node.key,hi)
        return left+[node.key]+right,1+max(lh,rh)
    ordered,height=walk(tree.root,None,float('-inf'),float('inf'))
    assert ordered==sorted(expected)
    for i,key in enumerate(ordered):
        node=seen[key]
        assert tree.search(key) is node
        successor,_=neighbor(node)
        predecessor,_=neighbor(node,False)
        assert successor is (seen[ordered[i+1]] if i+1<len(ordered) else None)
        assert predecessor is (seen[ordered[i-1]] if i else None)
        current=node
        total=0
        for k in range(1,len(ordered)-i+1):
            current,cost=neighbor(current)
            total+=cost
            assert total<=4*k+3*height+5
            if current is None:
                break
    return seen,height


def height(node):
    return -1 if node is None else 1+max(height(node.left),height(node.right))


def main():
    deletions=0
    events=set()
    for n in range(1,7):
        for order in permutations(range(n)):
            for key in range(n):
                tree=Tree(order)
                before,_=validate(tree,set(range(n)))
                tree.delete(before[key])
                after,_=validate(tree,set(range(n))-{key})
                assert all(after[k] is before[k] for k in after)
                events|=tree.events
                deletions+=1
    keys=[20,10,40,5,15,30,50,25,35,27]
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binary search trees solutions (continued)

```
    tree=Tree(keys)
    before,_=validate(tree,set(keys))
    assert neighbor(before[20])[0] is before[25]
    assert neighbor(before[27])[0] is before[30]
    tree.delete(before[20])
    after,_=validate(tree,set(keys)-{20})
    assert tree.root is before[25] and tree.root.parent is None
    assert after[30].left is before[27] and after[27].parent is before[30]
    assert after[25].right is before[40] and after[40].parent is before[25]
    events|=tree.events
    rng=random.Random(20261024)
    tree=Tree()
    reference=set()
    for _ in range(2000):
        key=rng.randrange(70)
        if rng.randrange(2):
            tree.insert(key)
            reference.add(key)
        elif key in reference:
            tree.delete(tree.search(key))
            reference.remove(key)
        validate(tree,reference)
        events|=tree.events
    for key in sorted(reference):
        tree.delete(tree.search(key))
    validate(tree,set())
    assert events=={'leaf','only-right','only-left','two-deep','two-direct','deep-successor-
has-right'}
    moments=[]
    permutation_count=0
    for n in range(1,9):
        heights=[height(Tree(order).root) for order in permutations(range(n))]
        expected_height=Fraction(sum(heights),len(heights))
        expected_exponential=Fraction(sum(2**h for h in heights),len(heights))
        assert expected_exponential<=Fraction(comb(n+3,3),2)
        permutation_count+=len(heights)
        moments.append((n,str(expected_height),str(expected_exponential)))
    assert moments[2][1]=='5/3'
    for n in range(1,80):
        b=[Fraction(comb(i+3,3),2) for i in range(n+1)]
        assert Fraction(4,n)*sum(b[:-1])==b[-1]
    print(f"PASS BST_single_deletions={deletions} mixed_operations=2000
all_parent_global_bounds_identity_and_neighbors=True events={sorted(events)}
height_permutations={permutation_count} exact_moments={moments}")


if __name__=='__main__':
    main()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Binary search trees solutions (continued)

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study